

Apuntes Completos: Práctica 0 y 1 de Programación II (ESEI - UVigo)

Índice

PARTE A: PRÁCTICA 0 — Sintaxis Java Procedural

1. Configuración inicial del proyecto
2. Ejercicio 1: Entrada/salida + Text Blocks
3. Ejercicio 2: Métodos y operaciones aritméticas
4. Ejercicio 3: Condicionales (if/else)
5. Ejercicio 4: switch + manejo de errores
6. Ejercicio 5: Bucles con validación (do-while)
7. Ejercicio 6: Algoritmos (números primos)
8. Ejercicio 7: Bucles envolventes
9. Ejercicio 8: Arrays estáticos
10. Ejercicio 9: Arrays dinámicos + métodos
11. Ejercicios 10-11: Integración con AEDI-Activities
12. Ejercicio 12: Matrices bidimensionales

PARTE B: PRÁCTICA 1 — Clases y Objetos

13. Fundamentos de POO aplicados a los ejercicios
14. Ejercicio 1: Clase Punto
15. Ejercicio 2: Clase Correo
16. Ejercicio 3: Clase Libro
17. Ejercicio 4: Sobrecarga + JUnit 5
18. Ejercicio 5: Clase Vehículo

Configuración inicial del proyecto

Paso 1: Crear proyecto Maven en VS Code

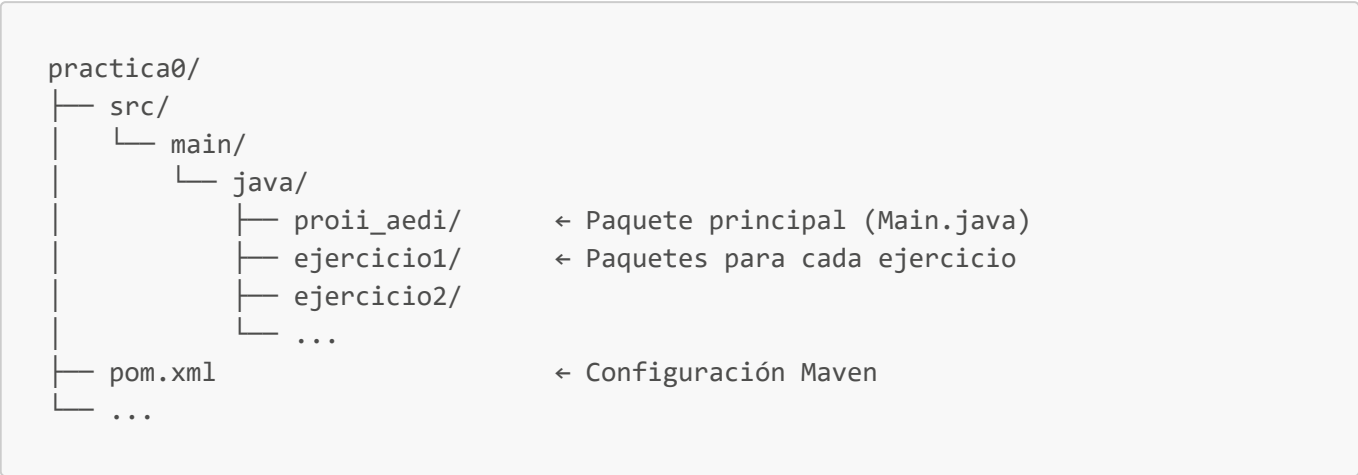
```
# Desde PowerShell (opcional, VS Code lo hace gráficamente)
mkdir C:\Users\Alastor\Documents\practica0
cd C:\Users\Alastor\Documents\practica0
```

En VS Code:

1. **Ctrl+Shift+P** → **"Java: Create Java Project"**
2. Selecciona **"Maven"**
3. groupId: `proii_aedi`
4. ArtifactId: `practica0`

- 5. Carpeta destino: `C:\Users\Alastor\Documents\practica0`
- 6. **NO uses arquetipo** → "No Archetype"

Paso 2: Estructura esperada



💡 **Tip clave:** Si no ves "Java Projects" en el sidebar:

- Reinicia VS Code
- Ejecuta `Ctrl+Shift+P` → "Java: Clean Java Language Server Workspace"

 Ejercicio 1: Entrada/salida + Text Blocks

Conceptos clave

Concepto	Explicación	Ejemplo
<code>System.out.println()</code>	Salida estándar con salto de línea	<code>println("Hola")</code>
Text Blocks	Texto multilínea (Java 15+)	<code>"""línea1\nlínea2"""</code>
<code>Scanner</code>	Lectura por teclado	<code>new Scanner(System.in)</code>
<code>nextLine()</code> / <code>nextInt()</code>	Métodos para leer tipos	<code>scanner.nextLine()</code>

Código completo (`src/main/java/ejercicio1/Ejercicio1.java`)

```
package ejercicio1;

import java.util.Scanner; // Importa la clase Scanner

public class Ejercicio1 {
    public static void main(String[] args) {
        // Parte 1: Mensaje simple
        System.out.println("Aprendiendo Java");

        // Parte 2: Text block (Java 15+)
        String mensaje = """
            Este es mi primer programa en Java
        """
    }
}
```

```

        con texto multilínea (text block)
        creado en la ESEI
        """;
    System.out.println(mensaje);

    // Parte 3: Lectura por teclado
    Scanner scanner = new Scanner(System.in); // Crea objeto Scanner

    System.out.print("Introduce tu nombre: ");
    String nombre = scanner.nextLine(); // Lee línea completa

    System.out.print("Introduce tu edad: ");
    int edad = scanner.nextInt(); // Lee entero

    System.out.print("Introduce tu nota: ");
    double nota = scanner.nextDouble(); // Lee decimal

    // Salida formateada
    System.out.println("\n=== DATOS INTRODUCIDOS ===");
    System.out.println("Nombre: " + nombre);
    System.out.println("Edad: " + edad + " años");
    System.out.printf("Nota: %.2f\n", nota); // %.2f = 2 decimales, %n = salto
de línea

    scanner.close(); // ¡Libera recursos! (buena práctica)
}
}

```

🔑 Errores comunes y soluciones

Error	Causa	Solución
<code>NoSuchElementException</code>	<code>nextInt()</code> deja <code>\n</code> en buffer	Usa <code>scanner.nextLine()</code> después de <code>nextInt()</code> para consumir el salto
Text block no funciona	Java < 15	Verifica versión: <code>java -version</code> → debe ser ≥ 15
<code>Scanner</code> no importado	Falta <code>import java.util.Scanner;</code>	Añade la línea de import al inicio

💡 **Relación con POO:** `Scanner` es una **clase** → `scanner` es un **objeto/instancia** de esa clase. Ya estás usando POO sin darte cuenta.

÷ Ejercicio 2: Métodos y operaciones aritméticas

Conceptos clave

Concepto	Explicación	Ejemplo
Método	Bloque de código reutilizable	<code>int suma(int a, int b) { ... }</code>

Concepto	Explicación	Ejemplo
<code>var</code>	Inferencia de tipo (Java 10+)	<code>var x = 5; → int x = 5;</code>
<code>return</code>	Devuelve valor al llamador	<code>return a + b;</code>
Validación	Control de errores	<code>if (divisor == 0) ...</code>

Código completo ([src/main/java/ejercicio2/Ejercicio2.java](#))

```
package ejercicio2;

public class Ejercicio2 {
    public static void main(String[] args) {
        var num1 = 5; // Inferencia: int num1 = 5;
        var num2 = 2;

        System.out.println("Número 1: " + num1);
        System.out.println("Número 2: " + num2);
        System.out.println("Suma: " + suma(num1, num2));
        System.out.println("Resta: " + resta(num1, num2));
        System.out.println("Multiplicación: " + multiplicacion(num1, num2));
        System.out.println("División: " + division(num1, num2));
    }

    // Método para suma
    public static int suma(int n1, int n2) {
        return n1 + n2;
    }

    // Método para resta
    public static int resta(int n1, int n2) {
        return n1 - n2;
    }

    // Método para multiplicación
    public static int multiplicacion(int n1, int n2) {
        return n1 * n2;
    }

    // Método para división con validación
    public static int division(int n1, int n2) {
        if (n2 == 0) {
            System.out.println("⚠ División por cero → resultado = 0");
            return 0; // Requisito del ejercicio
        }
        return n1 / n2;
    }
}
```

```
Llamada: suma(5, 2)
      ↓
Parámetros: n1=5, n2=2
      ↓
Ejecución: return 5 + 2
      ↓
Resultado: 7 → devuelto al llamador
```

💡 **Relación con POO:** Los métodos son el **comportamiento** de los objetos. En POO, estos métodos pertenecerán a clases específicas (ej: `Calculadora.suma()`).

12 34 Ejercicio 3: Condicionales (`if/else`)

Conceptos clave

Operador	Significado	Ejemplo
<code>%</code>	Módulo (resto división)	<code>7 % 2 = 1</code>
<code>==</code>	Igualdad	<code>x == 0</code>
<code>!=</code>	Distinto	<code>x != 0</code>
<code>> / <</code>	Mayor/menor	<code>x > 100</code>

Código completo (`src/main/java/ejercicio3/Ejercicio3.java`)

```
package ejercicio3;

import java.util.Scanner;

public class Ejercicio3 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Introduce un número entero: ");
        int numero = scanner.nextInt();

        // 1. Par/Impar (usa módulo %)
        if (numero % 2 == 0) {
            System.out.println("Número par");
        } else {
            System.out.println("Número impar");
        }

        // 2. Cero/No cero
        if (numero == 0) {
            System.out.println("Número cero");
        } else {
```

```

        System.out.println("Número distinto de cero");
    }

    // 3. Elevado/Bajo
    if (numero > 100) {
        System.out.println("Número elevado");
    } else {
        System.out.println("Número bajo");
    }

    scanner.close();
}
}

```

🔑 Tabla de verdad para par/impar

Número	numero % 2	Resultado
4	0	Par
7	1	Impar
-6	0	Par
0	0	Par

💡 **Optimización:** Para verificar par/impar, `numero % 2 == 0` es más eficiente que `numero / 2 * 2 == numero`.

⚙️ Ejercicio 4: `switch` + manejo de errores

Conceptos clave

Concepto	Explicación	Ejemplo
<code>switch</code>	Evaluación múltiple	<code>switch(op) { case '+': ... }</code>
<code>Double.NaN</code>	Valor especial "no es número"	<code>Double.NaN</code>
Validación	Prevenir errores en runtime	<code>if (divisor == 0) ...</code>

Código completo (`src/main/java/ejercicio4/Ejercicio4.java`)

```

package ejercicio4;

import java.util.Scanner;

public class Ejercicio4 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        double resultado = 0.0;
    }
}

```

```

System.out.print("Introduce operador (+, -, *, /): ");
char operador = scanner.next().charAt(0); // Lee primer carácter

System.out.print("Primer número: ");
double num1 = scanner.nextDouble();

System.out.print("Segundo número: ");
double num2 = scanner.nextDouble();

// Evaluación con switch (Java 14+)
switch (operador) {
    case '+':
        resultado = num1 + num2;
        break;
    case '-':
        resultado = num1 - num2;
        break;
    case '*':
        resultado = num1 * num2;
        break;
    case '/':
        if (num2 == 0) {
            System.out.println("La división no puede realizarse porque el
divisor es cero");
            resultado = Double.NaN; // Not a Number
        } else {
            resultado = num1 / num2;
        }
        break;
    default: // Caso no previsto
        System.out.println("Opción incorrecta");
        resultado = Double.NaN;
}

System.out.println("Resultado: " + resultado);
scanner.close();
}
}

```

Flujo de switch

```

operador = '/'
    ↓
¿Es '+'? → No
¿Es '-'? → No
¿Es '*'? → No
¿Es '/'? → Sí → Ejecuta bloque '/'
    ↓
break → Sale del switch

```

💡 **Double.NaN**: Valor especial definido por IEEE 754 para operaciones inválidas. `NaN != NaN` (¡curioso!).

📝 Ejercicio 5: Bucles con validación (**do-while**)

Conceptos clave

Bucle	Cuándo usar	Ejemplo
<code>for</code>	Número conocido de iteraciones	<code>for (int i=0; i<10; i++)</code>
<code>while</code>	Condición antes de ejecutar	<code>while (cond) { ... }</code>
<code>do-while</code>	Ejecutar al menos una vez	<code>do { ... } while (cond);</code>

Código completo (`src/main/java/ejercicio5/Ejercicio5.java`)

```
package ejercicio5;

import java.util.Scanner;

public class Ejercicio5 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int n;

        // Validación obligatoria: N debe ser positivo
        do {
            System.out.print("Introduce N (número positivo): ");
            n = scanner.nextInt();
            if (n <= 0) {
                System.out.println("❌ Error: N debe ser positivo. Inténtalo de nuevo.");
            }
        } while (n <= 0); // Repite MIENTRAS n no sea válido

        int suma = 0;
        int contador = 0;
        int numero = 2; // Primer número par

        // Suma los N primeros pares
        while (contador < n) {
            suma += numero; // Acumula
            numero += 2;    // Siguiendo par
            contador++;     // Contador
        }

        System.out.println("Suma de los primeros " + n + " números pares: " + suma);
        scanner.close();
    }
}
```



```
    }
}
```

Algoritmo visual

```
N = 3
Iteración 1: suma=0+2=2, numero=4, contador=1
Iteración 2: suma=2+4=6, numero=6, contador=2
Iteración 3: suma=6+6=12, numero=8, contador=3 → ¡fin!
Resultado: 12 (2+4+6)
```

 **Optimización matemática:** La suma de los N primeros pares = $N(N+1)$. Pero el ejercicio pide bucle → práctica obligatoria.

Ejercicio 6: Algoritmos (números primos)

Conceptos clave

Concepto	Explicación	Optimización
Número primo	Divisible solo por 1 y sí mismo	Descartar pares > 2
Divisibilidad	$n \% i == 0 \rightarrow$ divisible	Probar hasta $\sqrt{n}$
Early return	Salir temprano al encontrar divisor	<code>return false</code> inmediato

Código completo ([src/main/java/ejercicio6/Ejercicio6.java](#))

```
package ejercicio6;

import java.util.Scanner;

public class Ejercicio6 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Introduce un número entero (>1): ");
        int numero = scanner.nextInt();

        if (esPrimo(numero)) {
            System.out.println(numero + " es un número primo.");
        } else {
            System.out.println(numero + " NO es un número primo.");
        }

        scanner.close();
    }
}
```

```
// Método reutilizable para verificar primalidad
public static boolean esPrimo(int n) {
    if (n <= 1) return false;      // 0 y 1 no son primos
    if (n == 2) return true;       // 2 es primo (único par)
    if (n % 2 == 0) return false;  // Descartar pares > 2

    // Solo probamos divisores impares hasta √n
    for (int i = 3; i * i <= n; i += 2) {
        if (n % i == 0) {
            return false; // Encontró divisor → no es primo
        }
    }
    return true; // No encontró divisores → es primo
}
```

Demostración de optimización

```
n = 101
√101 ≈ 10.05 → probamos divisores hasta 10

Divisores probados: 3, 5, 7, 9
101 % 3 = 2 → no divisible
101 % 5 = 1 → no divisible
101 % 7 = 3 → no divisible
101 % 9 = 2 → no divisible
→ ¡101 es primo!
```

 **Relación con ciberseguridad:** Los primos grandes son la base de RSA. Este algoritmo es el punto de partida para entender criptografía asimétrica.

Ejercicio 7: Bucles envolventes

Conceptos clave

Concepto	Explicación	Ejemplo
Bucle envolvente	Bucle que repite todo el programa	<code>do { ... } while (seguir);</code>
Delegación	Separar lógica en métodos	<code>calcular(op, a, b)</code>
Reutilización	Usar código de ejercicio anterior	Copiar método <code>calcular</code>

Código completo ([src/main/java/ejercicio7/Ejercicio7.java](#))

```
package ejercicio7;

import java.util.Scanner;
```

```

public class Ejercicio7 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        char respuesta;

        // Bucle envolvente: repite hasta que usuario diga 'n'
        do {
            System.out.print("\nOperador (+, -, *, /): ");
            char operador = scanner.next().charAt(0);

            System.out.print("Primer número: ");
            double num1 = scanner.nextDouble();

            System.out.print("Segundo número: ");
            double num2 = scanner.nextDouble();

            double resultado = calcular(operador, num1, num2);
            System.out.println("Resultado: " + resultado);

            // Preguntar si continuar
            System.out.print("\n¿Otra operación? (s/n): ");
            respuesta = scanner.next().charAt(0);

        } while (respuesta == 's' || respuesta == 'S'); // Ignora
        mayúsculas/minúsculas

        System.out.println("¡Hasta luego!");
        scanner.close();
    }

    // Método reutilizado del Ejercicio 4
    public static double calcular(char operador, double num1, double num2) {
        switch (operador) {
            case '+': return num1 + num2;
            case '-': return num1 - num2;
            case '*': return num1 * num2;
            case '/':
                if (num2 == 0) {
                    System.out.println("⚠ División por cero");
                    return Double.NaN;
                }
                return num1 / num2;
            default:
                System.out.println("⚠ Operador inválido");
                return Double.NaN;
        }
    }
}

```

💡 **Patrón de diseño:** Este es un ejemplo temprano del patrón **"Template Method"** → estructura general del programa con pasos específicos delegados a métodos.

ID Ejercicio 8: Arrays estáticos

Conceptos clave

Concepto	Explicación	Ejemplo
Array estático	Tamaño fijo definido en compilación	<code>char[] letras = {...};</code>
Índice	Posición en array (0-based)	<code>letras[0] = 'T'</code>
Módulo %	Para mapear a rango válido	<code>dni % 23 → 0..22</code>

Código completo ([src/main/java/ejercicio8/Ejercicio8.java](#))

```
package ejercicio8;

import java.util.Scanner;

public class Ejercicio8 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Introduce tu número de DNI (8 dígitos): ");
        int dni = scanner.nextInt();

        char letra = calcularLetraDNI(dni);
        System.out.println("Tu DNI completo es: " + dni + letra);

        scanner.close();
    }

    public static char calcularLetraDNI(int dni) {
        // Array estático con letras según posición (0-22)
        char[] letras = {
            'T', 'R', 'W', 'A', 'G', 'M', 'Y', 'F', 'P', 'D',
            'X', 'B', 'N', 'J', 'Z', 'S', 'Q', 'V', 'H', 'L',
            'C', 'K', 'E'
        };

        int resto = dni % 23; // Siempre entre 0 y 22
        return letras[resto]; // Acceso directo por índice
    }
}
```

🔑 Tabla de mapeo DNI

Resto	Letra	Resto	Letra
0	T	12	N

Resto	Letra	Resto	Letra
1	R	13	J
2	W	14	Z
...	...	...	...
22	E		

💡 **¿Por qué 23?** 23 es primo → distribución uniforme de restos para evitar colisiones frecuentes.

Ejercicio 9: Arrays dinámicos + métodos

Conceptos clave

Concepto	Explicación	Ejemplo
Array dinámico	Tamaño definido en runtime	<code>new int[10]</code>
<code>length</code>	Propiedad del array	<code>array.length</code>
For-each	Iteración simplificada	<code>for (int num : array)</code>
Modularidad	Separar responsabilidades	<code>calcularMaximo(array)</code>

Código completo ([src/main/java/ejercicio9/Ejercicio9.java](#))

```
package ejercicio9;

import java.util.Scanner;

public class Ejercicio9 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int[] numeros = new int[10]; // Array de 10 enteros

        // Lectura de valores
        System.out.println("Introduce 10 números enteros:");
        for (int i = 0; i < 10; i++) {
            System.out.print("Número " + (i + 1) + ": ");
            numeros[i] = scanner.nextInt();
        }

        // Cálculos mediante métodos especializados
        int max = calcularMaximo(numeros);
        int min = calcularMinimo(numeros);
        double promedio = calcularPromedio(numeros);

        // Resultados
        System.out.println("\n=== RESULTADOS ===");
        System.out.println("Máximo: " + max);
    }
}
```

```

        System.out.println("Mínimo: " + min);
        System.out.printf("Promedio: %.2f%n", promedio);

        scanner.close();
    }

    // Método para máximo
    public static int calcularMaximo(int[] array) {
        int max = array[0]; // Inicializa con primer elemento
        for (int i = 1; i < array.length; i++) {
            if (array[i] > max) max = array[i]; // Actualiza si encuentra mayor
        }
        return max;
    }

    // Método para mínimo
    public static int calcularMinimo(int[] array) {
        int min = array[0];
        for (int i = 1; i < array.length; i++) {
            if (array[i] < min) min = array[i];
        }
        return min;
    }

    // Método para promedio
    public static double calcularPromedio(int[] array) {
        int suma = 0;
        for (int num : array) { // For-each: más legible
            suma += num;
        }
        return (double) suma / array.length; // Cast a double para división real
    }
}

```

Diagrama de memoria para arrays

```

numeros (referencia en STACK)
    ↓
[5, 3, 8, 1, 9, 2, 7, 4, 6, 0] ← Array en HEAP
  ↑ ↑ ↑
[0][1][2] → Índices (0-based)

```

 **Error común:** Olvidar el cast (`double`) en promedio → división entera ($5/2 = 2$ en lugar de 2.5).

Ejercicios 10-11: Integración con AEDI-Activities

Requisitos previos

1. Descargar [AEDI-Activities.zip](#) del campus virtual de la asignatura

2. Descomprimir en `C:\Users\Alastor\Documents\AEDI-Activities`

Configuración en VS Code

1. `File` → `Add Folder to Workspace` → selecciona `AEDI-Activities`
2. Tu workspace debe tener dos carpetas:

```
WORKSPACE
├── practica0
└── AEDI-Activities
```

Código para Ejercicio 10 (`src/main/java/ejercicio10/Ejercicio10.java`)

```
package ejercicio10;

import activity0.ArrayUtils; // Importa desde proyecto externo
import java.util.Scanner;

public class Ejercicio10 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int[] numeros = new int[10];

        System.out.println("Introduce 10 números:");
        for (int i = 0; i < 10; i++) {
            numeros[i] = scanner.nextInt();
        }

        // Usa métodos del paquete activity0
        int max = ArrayUtils.maximo(numeros);
        int min = ArrayUtils.minimo(numeros);
        double promedio = ArrayUtils.promedio(numeros);

        System.out.println("Máximo: " + max);
        System.out.println("Mínimo: " + min);
        System.out.printf("Promedio: %.2f\n", promedio);

        scanner.close();
    }
}
```

⚠ **Si no tienes acceso a AEDI-Activities:** Pide el proyecto a tu profesor/a. Es material oficial de la asignatura.

🔗 Ejercicio 12: Matrices bidimensionales

Conceptos clave

Concepto	Explicación	Ejemplo
Matriz 2D	Array de arrays	<code>int[][] matriz = new int[3][3]</code>
Doble bucle	Recorrer filas y columnas	<code>for (fila) for (col)</code>
<code>break</code> etiquetado	Salir de bucles anidados	<code>outer: for (...) { break outer; }</code>

Código completo ([src/main/java/ejercicio12/Ejercicio12.java](#))

```
package ejercicio12;

import java.util.Scanner;

public class Ejercicio12 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int[][] matriz = new int[3][3]; // Matriz 3x3

        // Lectura de la matriz
        System.out.println("Introduce 9 números para la matriz 3x3:");
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                System.out.print("matriz[" + i + "][" + j + "]: ");
                matriz[i][j] = scanner.nextInt();
            }
        }

        // Número a buscar
        System.out.print("\nIntroduce el número a buscar: ");
        int buscado = scanner.nextInt();

        // Búsqueda con detención temprana
        boolean encontrado = false;
        outer: // Etiqueta para salir de ambos bucles
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                if (matriz[i][j] == buscado) {
                    System.out.println("✅ Encontrado en fila " + i + ", columna " + j);
                    encontrado = true;
                    break outer; // Sale inmediatamente de ambos bucles
                }
            }
        }

        if (!encontrado) {
            System.out.println("❌ El número " + buscado + " no está en la matriz.");
        }

        scanner.close();
    }
}
```



```
    }  
}
```

🔑 Representación visual de matriz

```
matriz[0][0]  matriz[0][1]  matriz[0][2]  ← Fila 0  
matriz[1][0]  matriz[1][1]  matriz[1][2]  ← Fila 1  
matriz[2][0]  matriz[2][1]  matriz[2][2]  ← Fila 2  
  ↑           ↑           ↑  
Columna 0    Columna 1    Columna 2
```

💡 **Alternativa sin etiqueta:** Usar bandera booleana en condición del bucle (`for (int i=0; i<3 && !encontrado; i++)`).

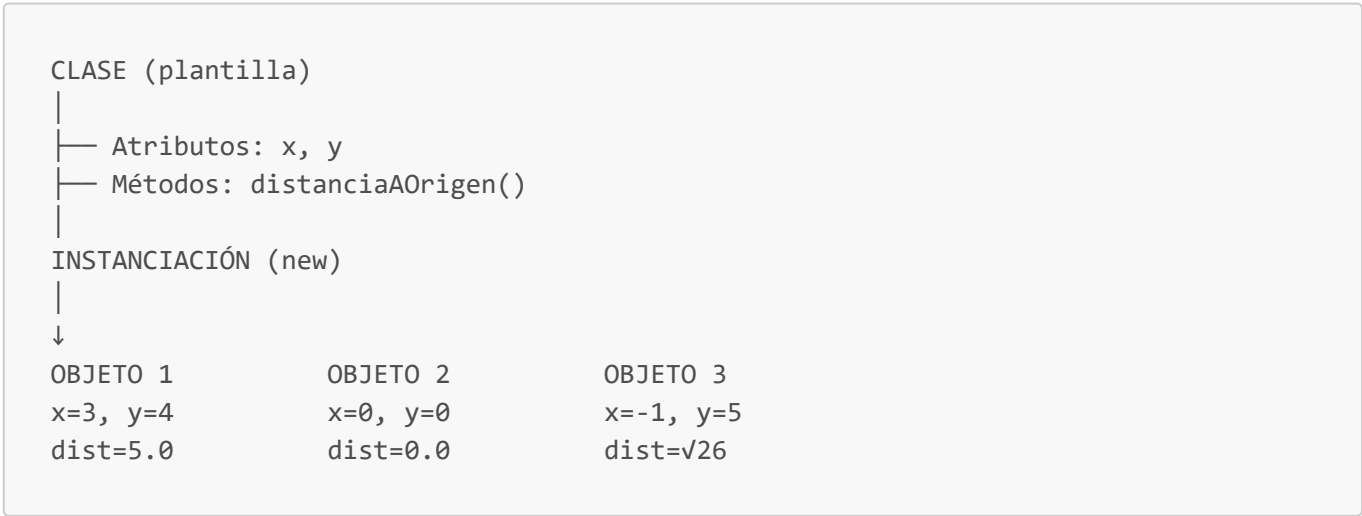
📦 PARTE B: PRÁCTICA 1 — Clases y Objetos

Fundamentos de POO aplicados a los ejercicios

Las 4 pilares de la POO (relacionados con ejercicios)

Pilar	Ejercicio donde se aplica	Ejemplo concreto
Abstracción	Ej. 1 (Punto)	<code>p.distanciaAOrigen()</code> oculta cálculo $\sqrt{x^2+y^2}$
Encapsulamiento	Ej. 5 (Vehículo)	Atributos <code>private</code> + métodos <code>public</code>
Herencia	No en Práctica 1	<code>Coche extends Vehiculo</code> (Práctica 2)
Polimorfismo	Ej. 4 (Calculadora)	Sobrecarga de métodos <code>sumar()</code>

Clase vs Objeto vs Instancia (visual)



📍 Ejercicio 1: Clase `Punto`

Diagrama UML simplificado

Código completo ([src/main/java/ejercicio1/Punto.java](#))

```

package ejercicio1;

public class Punto {
    // Atributos con visibilidad por defecto (package-private)
    double x;
    double y;

    // Constructor con parámetros
    public Punto(double x, double y) {
        this.x = x; // this.x = atributo | x = parámetro
        this.y = y;
    }

    // Constructor sin parámetros (opcional)
    public Punto() {
        this.x = 0.0;
        this.y = 0.0;
    }

    // Método: distancia al origen (0,0)
    public double distanciaAOrigen() {
        return Math.sqrt(x * x + y * y); // this implícito
    }

    // Método: distancia a otro punto
    public double distanciaA(Punto otro) {
        double dx = this.x - otro.x;
        double dy = this.y - otro.y;
        return Math.sqrt(dx * dx + dy * dy);
    }

    // Método toString() para visualización amigable
    @Override
    public String toString() {
        return "(" + x + ", " + y + ")";
    }
}

```

```

    }
}

```

Código de prueba ([src/main/java/ejercicio1/Ejercicio1.java](#))

```

package ejercicio1;

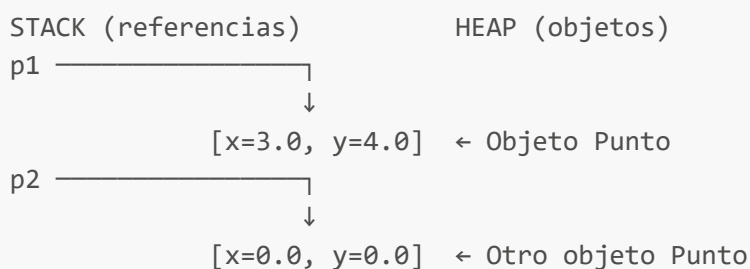
public class Ejercicio1 {
    public static void main(String[] args) {
        // Crear instancias (objetos)
        Punto p1 = new Punto(3.0, 4.0); // this.x=3, this.y=4
        Punto p2 = new Punto(0.0, 0.0);
        Punto p3 = new Punto(); // Usa constructor sin parámetros

        // Usar métodos
        System.out.println("Punto 1: " + p1); // Llama implícitamente a toString()
        System.out.println("Distancia al origen: " + p1.distanciaAOrigen()); // →
5.0
        System.out.println("Distancia a p2: " + p1.distanciaA(p2)); // → 5.0

        // Modificar atributos directamente (visibilidad package-private lo
permite)
        p3.x = 1.0;
        p3.y = 1.0;
        System.out.println("Punto 3 modificado: " + p3);
    }
}

```

🔑 Memoria: ¿Dónde se almacenan?



💡 **this explicado:** Es una referencia **implícita** al objeto actual. En `p1.distanciaAOrigen()`, `this` apunta a `p1`.

✉ Ejercicio 2: Clase Correo

Diagrama UML

```

+-----+
|          Correo          |
+-----+
| - nombre: String         |
| - apellidos: String      |
| - usuario: String        |
| - servidor: String       |
+-----+
| + Correo(nom, ap, usu, serv) |
| + Correo(nom, ap)          | ← Sobrecarga de constructor
| + toString(): String       |
+-----+

```

Código completo ([src/main/java/ejercicio2/Correo.java](#))

```

package ejercicio2;

public class Correo {
    private String nombre;
    private String apellidos;
    private String usuario;
    private String servidor;

    // Constructor completo (4 parámetros)
    public Correo(String nombre, String apellidos, String usuario, String
servidor) {
        this.nombre = nombre;
        this.apellidos = apellidos;
        this.usuario = usuario;
        this.servidor = servidor;
    }

    // Constructor simplificado (2 parámetros) - SOBRECARGA
    public Correo(String nombre, String apellidos) {
        this.nombre = nombre;
        this.apellidos = apellidos;

        // Construir usuario: primer apellido + inicial del nombre
        String[] partesApellidos = apellidos.split(" ");
        String primerApellido = partesApellidos[0];
        String inicialNombre = nombre.substring(0, 1).toLowerCase();
        this.usuario = primerApellido.toLowerCase() + inicialNombre;

        // Servidor fijo
        this.servidor = "esei.uvigo.es";
    }

    // Método toString() según especificación
    @Override
    public String toString() {

```

```

        return apellidos + ", " + nombre + ": " + usuario + "@" + servidor;
    }
}

```

Código de prueba ([src/main/java/ejercicio2/Ejercicio2.java](#))

```

package ejercicio2;

public class Ejercicio2 {
    public static void main(String[] args) {
        // Constructor completo
        Correo c1 = new Correo("Ana", "García López", "agarcia", "gmail.com");
        System.out.println(c1); // → García López, Ana: agarcia@gmail.com

        // Constructor simplificado
        Correo c2 = new Correo("Carlos", "Fernández Rodríguez");
        System.out.println(c2); // → Fernández Rodríguez, Carlos:
fernandezc@esei.uvigo.es
    }
}

```

Sobrecarga de constructores

```

Correo(String, String, String, String) ← Versión 1
Correo(String, String)                  ← Versión 2 (sobrecarga)
    ↑
Mismo nombre, diferentes parámetros → el compilador elige según contexto

```

 **Método `split()`**: Divide cadena usando expresión regular. `"García López".split(" ")` → `["García", "López"]`.

Ejercicio 3: Clase **Libro**

Código completo ([src/main/java/ejercicio3/Libro.java](#))

```

package ejercicio3;

public class Libro {
    private String titulo;
    private String autores;
    private String editorial;
    private int año;
    private String isbn;

    // Constructor

```

```
public Libro(String titulo, String autores, String editorial, int año, String isbn) {
    this.titulo = titulo;
    this.autores = autores;
    this.editorial = editorial;
    this.año = año;
    this.isbn = isbn;
}

// Método toString() con formato especificado
@Override
public String toString() {
    return ""
        + Titulo = %s
        + Autores = %s
        + Editorial = %s
        + Año = %d
        + Isbn = %s
        + "".formatted(titulo, autores, editorial, año, isbn);
}

// Getters (acceso controlado a atributos)
public String getTitulo() { return titulo; }
public String getAutores() { return autores; }
public String getEditorial() { return editorial; }
public int getAño() { return año; }
public String getIsbn() { return isbn; }
}
```

Código de prueba (src/main/java/ejercicio3/Ejercicio3.java)

```
package ejercicio3;

import java.util.Scanner;

public class Ejercicio3 {
    public static void main(String[] args) {
        // Crear libro directamente
        Libro libro1 = new Libro(
            "Thinking in Java",
            "Bruce Eckel",
            "Prentice Hall",
            2007,
            "0131872486"
        );
        System.out.println(libro1);

        // Crear libro pidiendo datos por teclado (ampliación opcional)
        Libro libro2 = crearLibroPorTeclado();
        System.out.println("\nLibro creado por teclado:");
        System.out.println(libro2);
    }
}
```

```

    }

    // Método auxiliar para crear libro por teclado
    public static Libro crearLibroPorTeclado() {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Título: ");
        String titulo = scanner.nextLine();

        System.out.print("Autores: ");
        String autores = scanner.nextLine();

        System.out.print("Editorial: ");
        String editorial = scanner.nextLine();

        System.out.print("Año: ");
        int año = scanner.nextInt();
        scanner.nextLine(); // Consumir salto de línea pendiente

        System.out.print("ISBN: ");
        String isbn = scanner.nextLine();

        scanner.close();
        return new Libro(titulo, autores, editorial, año, isbn);
    }
}

```

Text Blocks + `formatted()`

```

// Text block (Java 15+)
String tb = """
    línea 1
    línea 2
    """;

// formatted() (Java 15+)
String f = "Hola %s".formatted("mundo"); // → "Hola mundo"

```

 **Ventaja:** Text blocks preservan sangría y saltos de línea → ideal para plantillas, SQL, JSON.

+ Ejercicio 4: Sobrecarga + JUnit 5

Código de la clase (`src/main/java/ejercicio4/Calculadora.java`)

```

package ejercicio4;

public class Calculadora {
    // Sobrecarga 1: sumar dos enteros

```

```
public int sumar(int a, int b) {
    return a + b;
}

// Sobrecarga 2: sumar tres enteros
public int sumar(int a, int b, int c) {
    return a + b + c;
}

// Sobrecarga 3: sumar dos doubles
public double sumar(double a, double b) {
    return a + b;
}

// Sobrecarga 4: multiplicar dos enteros
public int multiplicar(int a, int b) {
    return a * b;
}

// Sobrecarga 5: multiplicar entero por double
public double multiplicar(int a, double b) {
    return a * b;
}

// Sobrecarga 6: descuento porcentual
public double aplicarDescuento(double precio, double porcentaje) {
    return precio * (1 - porcentaje / 100);
}

// Sobrecarga 7: descuento fijo
public double aplicarDescuento(double precio, double descuentoFijo) {
    return precio - descuentoFijo;
}

// Sobrecarga 8: descuento porcentual con límite
public double aplicarDescuento(double precio, double porcentaje, double
maxDescuento) {
    double descuento = precio * (porcentaje / 100);
    descuento = Math.min(descuento, maxDescuento); // Límite máximo
    return precio - descuento;
}
}
```

Código de test ([src/test/java/ejercicio4/CalculadoraTest.java](#))

```
package ejercicio4;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class CalculadoraTest {
```



```

private Calculadora calc = new Calculadora();

@Test
void testSumarDosEnteros() {
    assertEquals(5, calc.sumar(2, 3));
}

@Test
void testSumarTresEnteros() {
    assertEquals(10, calc.sumar(2, 3, 5));
}

@Test
void testSumarDosDoubles() {
    assertEquals(5.5, calc.sumar(2.5, 3.0), 0.001); // Delta para comparar
doubles
}

@Test
void testMultiplicarEnteroDouble() {
    assertEquals(7.5, calc.multiplicar(3, 2.5), 0.001);
}

@Test
void testDescuentoPorcentual() {
    assertEquals(90.0, calc.aplicarDescuento(100.0, 10.0), 0.001);
}

@Test
void testDescuentoConLimite() {
    // 20% de 100 = 20, pero límite es 15 → descuento = 15
    assertEquals(85.0, calc.aplicarDescuento(100.0, 20.0, 15.0), 0.001);
}
}

```

Configuración Maven para JUnit 5 (pom.xml)

```

<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.10.0</version>
    <scope>test</scope>
  </dependency>
</dependencies>

```

 **Ejecutar tests en VS Code:** Clic derecho en `CalculadoraTest.java` → "Run Tests".

Ejercicio 5: Clase `Vehiculo`

Código completo (src/main/java/ejercicio5/Vehiculo.java)

```
package ejercicio5;

public class Vehiculo {
    private int maxPasajeros;
    private double capacidadDeposito; // litros
    private double consumoMedio;      // litros/100km

    // Constructor
    public Vehiculo(int maxPasajeros, double capacidadDeposito, double
consumoMedio) {
        this.maxPasajeros = maxPasajeros;
        this.capacidadDeposito = capacidadDeposito;
        this.consumoMedio = consumoMedio;
    }

    // Método: distancia con depósito lleno
    public double distanciaConDepositoLleno() {
        return (capacidadDeposito / consumoMedio) * 100;
    }

    // Método: distancia con litros específicos
    public double distanciaConLitros(double litros) {
        if (litros < 0 || litros > capacidadDeposito) {
            throw new IllegalArgumentException(
                "Litros debe estar entre 0 y " + capacidadDeposito
            );
        }
        return (litros / consumoMedio) * 100;
    }

    // Getters
    public int getMaxPasajeros() { return maxPasajeros; }
    public double getCapacidadDeposito() { return capacidadDeposito; }
    public double getConsumoMedio() { return consumoMedio; }

    // toString() para visualización
    @Override
    public String toString() {
        return "Vehículo: " + maxPasajeros + " pasajeros, " +
            capacidadDeposito + "L depósito, " +
            consumoMedio + "L/100km consumo";
    }
}
```

Código de prueba (src/main/java/ejercicio5/Ejercicio5.java)

```
package ejercicio5;
```

```

public class Ejercicio5 {
    public static void main(String[] args) {
        // Ejemplo del enunciado: 4 pasajeros, 60L, 6L/100km
        Vehiculo v = new Vehiculo(4, 60.0, 6.0);
        System.out.println(v);

        double distanciaLleno = v.distanciaConDepositoLleno();
        System.out.printf("Distancia con depósito lleno: %.1f km%n",
            distanciaLleno);
        // → (60 / 6) * 100 = 1000 km

        double distancia30L = v.distanciaConLitros(30.0);
        System.out.printf("Distancia con 30L: %.1f km%n", distancia30L);
        // → (30 / 6) * 100 = 500 km
    }
}

```

Fórmula matemática

$$\text{distancia} = (\text{litros} / \text{consumo}) * 100$$

Ejemplo:

litros = 60L

consumo = 6L/100km

$\text{distancia} = (60 / 6) * 100 = 10 * 100 = 1000 \text{ km}$

 **Validación en métodos:** `distanciaConLitros()` lanza excepción si litros fuera de rango → defensa contra errores del usuario.

☒ Checklist final de aprendizaje

Concepto	Práctica 0	Práctica 1	Dominio
Variables y tipos	☒ Ej. 1		
Condicionales	☒ Ej. 3		
Bucles	☒ Ej. 5,7		
Arrays	☒ Ej. 8,9,12		
Métodos	☒ Ej. 2,4		
Clases y objetos		☒ Ej. 1	
Constructores		☒ Ej. 1,2	
Sobrecarga		☒ Ej. 2,4	
Encapsulamiento		☒ Ej. 3,5	

Concepto	Práctica 0	Práctica 1	Dominio
<code>this</code>		☒ Ej. 1	
Text Blocks	☒ Ej. 1	☒ Ej. 3	

Próximos pasos recomendados

1. **Práctica 1 entregada** → Enfócate en:

- Herencia (`extends`)
- Interfaces (`implements`)
- Polimorfismo real (no solo sobrecarga)

2. **Para ciberseguridad** (tu interés):

- Clases inmutables (`final` + atributos `private final`)
- Validación de entradas en constructores (evitar inyecciones)
- Sobrescritura segura de `equals()` y `hashCode()`

3. **Herramientas avanzadas:**

- Debugger de VS Code para seguir flujo de ejecución
- Git para versionar tus prácticas (ya usas Winget/Git según tus memorias)